অধ্যায় ৮

Strings এবং Modern Text Processing

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- String লেখার তিনটি পদ্ধতি এবং Escape Sequence ব্যবহার করতে।
- `length` আসলে কী গোনে, এবং বাংলা লেখা ও Emoji-তে কেন তা চোখে দেখা অক্ষরের সংখ্যা নয়, তা ব্যাখ্যা করতে।
- `at()`, `slice()`, `includes()`, `indexOf()`, `startsWith()` ও `endsWith()` দিয়ে লেখার অংশ পড়তে ও খুঁজতে।
- `trim()`, `replace()`, `replaceAll()`, `padStart()` দিয়ে লেখা পরিষ্কার ও সাজাতে।
- `split()` ও `join()` দিয়ে লেখাকে শব্দে ভাগ করতে এবং আবার জোড়া লাগাতে।
- Unicode Normalization কেন প্রয়োজন, বিশেষত বাংলা লেখার তুলনায়, তা বুঝতে।
- Regular Expression দিয়ে সরল Pattern খুঁজতে, বদলাতে এবং যাচাই করতে।
- ব্যবহারকারীর লেখা Page-এ নিরাপদে দেখাতে এবং XSS-এর ঝুঁকি চিনতে।
- একটি সম্পূর্ণ **Content Search Utility** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭ সম্পন্ন করা: Object, `Object.entries()` এবং Destructuring।
- অধ্যায় ৬-এর Array Method: `map`, `filter`, `join`।
- অধ্যায় ১-এর `textContent` বনাম `innerHTML`-এর প্রাথমিক ধারণা।
- `chapter-08` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

ব্যবহারকারীর নাম, Email, ঠিকানা, খোঁজার শব্দ, একটি লেখার শিরোনাম, Server-এর বার্তা—Program-এ আসা তথ্যের বড় অংশই লেখা। প্রায় প্রতিটি Program-কে লেখা পরিষ্কার করতে, ভাগ করতে, খুঁজতে এবং যাচাই করতে হয়।

এর অনেকটাই সহজ, কিন্তু কিছু ফাঁদ আছে যা ইংরেজি-ভাষী প্রোগ্রামাররা প্রায়ই চোখে পান না। `'ক্ষমা'` -র `length` কত? দেখতে একই রকম দুটি বাংলা শব্দ কেন `===` দিয়ে সমান হয় না? কেন খোঁজার শব্দ লেখায় থাকা সত্ত্বেও Program তা খুঁজে পায় না? এই অধ্যায়ে এসব প্রশ্নের উত্তর আছে, কারণ বাংলা লেখা নিয়ে কাজ করা Program-এর জন্য এগুলো প্রান্তিক বিষয় নয়।

শেষে একটি গুরুত্বপূর্ণ নিরাপত্তার বিষয়: ব্যবহারকারীর লেখা যদি অসাবধানে Page-এ বসানো হয়, তার ভেতরের Code চলে যেতে পারে। ওয়েবের সবচেয়ে প্রচলিত আক্রমণগুলোর একটি এভাবেই ঘটে।

৮.১ String কী

String হলো অক্ষরের একটি ক্রম। এটি একটি Primitive (অধ্যায় ২), এবং এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য আছে: **String বদলানো যায় না** (Immutable)।

immutable.js · JavaScript — চলবে: Browser Console ও Node.js

```
let city = 'dhaka';

city[0] = 'D';
console.log(city);

const upper = city.toUpperCase();
console.log(upper);
console.log(city);

city = upper;
console.log(city);
```

Output

```
dhaka
DHAKA
dhaka
DHAKA
```

`city[0] = 'D'` কোনো কাজ করেনি, এবং Error-ও দেয়নি। `toUpperCase()` মূল String বদলায়নি; একটি নতুন String ফেরত দিয়েছে। এই অধ্যায়ের সব String Method এভাবেই কাজ করে: ফল চাইলে নতুন String-টি একটি Variable-এ রাখতে হয়। শেষ দুই লাইনে `city` নামটিকে নতুন String-এর দিকে সরানো হয়েছে; String নিজে বদলায়নি।

৮.২ String লেখা

String তিনভাবে লেখা যায়: Single Quote, Double Quote এবং Backtick (অধ্যায় ১)। Backtick-এর String-কে **Template Literal** বলে। এর দুটি বিশেষ সুবিধা: `${ }` দিয়ে মান বসানো, এবং একাধিক লাইনে লেখা।

template-literal.js · JavaScript — চলবে: Browser Console ও Node.js

```
const name = 'Sara';
const marks = 88;

console.log(`${name} পেয়েছেন ${marks}, Grade ${marks >= 80 ? 'A+' : 'A'}`);

const receipt = `দোকান: Boi Ghar
মোট: ${120 * 3} টাকা`;
console.log(receipt);
```

Output

```
Sara পেয়েছেন 88, Grade A+
দোকান: Boi Ghar
মোট: 360 টাকা
```

`${ }`-এর ভেতরে যেকোনো Expression বসানো যায়: Variable, হিসাব, এমনকি Ternary।

Escape Sequence

কিছু অক্ষর সরাসরি টাইপ করা যায় না, অথবা String-এর নিয়ম ভেঙে দেয়। সেগুলো লেখা হয় Backslash (`\`) দিয়ে শুরু হওয়া **Escape Sequence** দিয়ে।

escapes.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('প্রথম লাইন\nদ্বিতীয় লাইন');
console.log('It\'s a book');
console.log('C:\\Users\\Sara');
console.log('\u0995\u09BE');
```

Output

```
প্রথম লাইন
দ্বিতীয় লাইন
It's a book
C:\Users\Sara
কা
```

Escape	অর্থ
`\n`	নতুন লাইন

Escape	অর্থ
`\t`	Tab
`\'` এবং `\"`	String-এর ভেতরে Quotation Mark
`\\`	একটি Backslash
`\uXXXX`	Unicode নম্বর দিয়ে একটি অক্ষর

শেষ লাইনে `\u0995` হলো `ক` এবং `\u09BE` হলো `া`; একসঙ্গে `কা`। প্রতিটি অক্ষরের এমন একটি নম্বর আছে, এবং পরের অনুচ্ছেদে এই নম্বরগুলোই গুরুত্বপূর্ণ হয়ে উঠবে।

৮.৩ Length, Index এবং Unicode

`length` মনে হয় অক্ষরের সংখ্যা দেয়। ইংরেজিতে সাধারণত তা-ই। বাংলায় এবং Emoji-তে নয়।

length.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('Dhaka'.length);
console.log('ঢাকা'.length);
console.log('ক্ষমা'.length);

const smile = '\u{1F600}';

console.log(smile.length);
console.log([...smile].length);
```

Output

```
5
4
5
2
1
```

কেন এমন হয়, তা বুঝতে তিনটি স্তর আলাদা করতে হবে:

স্তর	কী	'ক্ষমা' -তে
Code Unit	JavaScript যা দিয়ে String রাখে (UTF-16)	৫
Code Point	Unicode-এর একটি অক্ষর-নম্বর	৫: ক, ◌্, ষ, ম, ◌া

স্তর	কী	`'ক্ষমা'` -তে
Grapheme	পাঠকের চোখে একটি অক্ষর	২: ক্ষ, মা

- **`length` গোনে Code Unit।** বেশির ভাগ অক্ষর একটি Code Unit, কিন্তু Emoji-র মতো কিছু অক্ষর দুটি। তাই হাসিমুখ Emoji `smile`-এর `length` হলো `2`। (Emoji-টি এখানে Unicode নম্বর `\u{1F600}` দিয়ে লেখা; পাঁচ অঙ্কের নম্বরের জন্য `{ }` লাগে।)
- **`[...text]` গোনে Code Point।** Spread একটি String-কে Code Point ধরে ভাগ করে, তাই Emoji একটিই।
- **বাংলায় Code Point আর চোখে দেখা অক্ষর এক নয়।** `ক্ষ` তিনটি Code Point দিয়ে তৈরি: `ক`, হসন্ত (`্`) এবং `ষ`। `মা` দুটি: `ম` এবং আ-কার। কার-চিহ্ন, হসন্ত, চন্দ্রবিন্দু—সবই আলাদা Code Point।

চোখে দেখা অক্ষর গুনতে `Intl.Segmenter` ব্যবহার করুন। এটি আধুনিক Browser ও Node.js-এ আছে।

graphemes.js · JavaScript — চলবে: Browser Console ও Node.js

```
const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
const text = 'ক্ষমা';
const letters = [...segmenter.segment(text)].map((part) => part.segment);

console.log(letters.length);
console.log(letters.join(' | '));
```

Output

```
2
ক্ষ | মা
```

কখন কোনটি দরকার? Database-এর একটি ঘরে কত জায়গা লাগবে—Code Unit (`length`)। "নাম কমপক্ষে ২ অক্ষর" জাতীয় নিয়ম, যা মানুষ পড়ে বিচার করবে—Grapheme। কোনো লেখা অক্ষর ধরে কাটতে হলে Grapheme-এর সীমায় কাটুন, নইলে একটি যুক্তাক্ষরের অর্ধেক বা একটি Emoji-র অর্ধেক কেটে অর্থহীন চিহ্ন তৈরি হয়।

৮.৪ অংশ পড়া: at এবং slice

String-এর Index ঠিক Array-র মতো শূন্য থেকে শুরু (অধ্যায় ৬)।

access.js · JavaScript — চলবে: Browser Console ও Node.js

```
const code = 'JS-2026-BD';

console.log(code[0]);
console.log(code.at(-1));
console.log(code.slice(3, 7));
console.log(code.slice(-2));
console.log(code.slice(0, 2));
```

Output

```
J
D
2026
BD
JS
```

`slice(start, end)` Array-র `slice`-এর মতোই: `start` থেকে, `end`-এর আগে পর্যন্ত। ঋণাত্মক সংখ্যা শেষ থেকে গোনে। পুরোনো Code-এ `substring()` ও `substr()` দেখা যায়; নতুন Code-এ `slice()` ব্যবহার করুন, কারণ এর আচরণ সবচেয়ে সামঞ্জস্যপূর্ণ।

> **সতর্কতা:** Index এবং `slice` Code Unit ধরে কাজ করে। `'ক্ষমা'.slice(0, 2)` দেয় `'ক্'`—একটি অসম্পূর্ণ যুক্তাক্ষর। বাংলা লেখা অক্ষর ধরে কাটতে হলে আগে `Intl.Segmenter` দিয়ে ভাগ করুন।

৮.৫ খোঁজা

search.js · JavaScript — চলবে: Browser Console ও Node.js

```
const title = 'JavaScript শেখার সহজ উপায়';

console.log(title.includes('শেখার'));
console.log(title.startsWith('Java'));
console.log(title.endsWith('উপায়'));
console.log(title.indexOf('Script'));
console.log(title.indexOf('Python'));
console.log(title.includes('javascript'));
```

Output

```
true
true
true
4
-1
false
```

Method	ফেরত দেয়
`includes(text)`	আছে কি না (`true` / `false`)
`startsWith(text)` / `endsWith(text)`	এ দিয়ে শুরু / শেষ কি না
`indexOf(text)`	প্রথম অবস্থানের Index; না পেলে `-1`

Method	ফেরত দেয়
`lastIndexOf(text)`	শেষ অবস্থানের Index; না পেলে `-1`

শেষ লাইনটি লক্ষ করুন: সব খোঁজা **Case-sensitive**। `'javascript'` আর `'JavaScript'` আলাদা। ব্যবহারকারীর খোঁজার শব্দ সাধারণত বড়-ছোট হাতের অক্ষর নির্বিশেষে মেলানো উচিত; তার উপায় অনুচ্ছেদ ৮.৭-এ।

`indexOf()`-এর দ্বিতীয় Argument দিয়ে বলা যায় কোন Index থেকে খোঁজা শুরু হবে। এটি দিয়ে একটি শব্দের সব অবস্থান খুঁজে বের করা যায়, যা Project-এ কাজে লাগবে।

৮.৬ পরিষ্কার করা ও সাজানো

clean.js · JavaScript — চলবে: Browser Console ও Node.js

```
const raw = '   Sara    Ahmed   ';

console.log(`[${raw.trim()}]`);
console.log(`[${raw.trimStart()}]`);

const price = '1,250,000';
console.log(price.replace(',', ''));
console.log(price.replaceAll(',', ''));

console.log('7'.padStart(3, '0'));
console.log('-'.repeat(10));
```

Output

```
[Sara    Ahmed]
[Sara    Ahmed   ]
1250,000
1250000
007
----------
```

- **`trim()`** শুধু শুরু ও শেষের ফাঁকা জায়গা সরায়; মাঝের বাড়তি ফাঁকা জায়গা থেকে যায়। `trimStart()` ও `trimEnd()` এক প্রান্তের জন্য।
- **`replace(a, b)`** শুধু **প্রথম** মিলটি বদলায়। এটি একটি প্রচলিত ভুলের উৎস।
- **`replaceAll(a, b)`** সব মিল বদলায়।
- **`padStart(length, fill)`** বাঁ দিকে অক্ষর বসিয়ে নির্দিষ্ট দৈর্ঘ্যে আনে: রোল নম্বর, রসিদ নম্বর বা সময় (`'07'`) দেখাতে কাজে লাগে। `padEnd()` ডান দিকে।
- **`repeat(n)`** String-কে `n` বার পুনরাবৃত্তি করে।

মাঝের বাড়তি ফাঁকা জায়গা সরাতে Regular Expression লাগে (অনুচ্ছেদ ৮.১০)।

৮.৭ Case Conversion এবং তুলনা

case.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log('Dhaka'.toUpperCase());
console.log('DHAKA'.toLowerCase());
console.log('ঢাকা'.toUpperCase());

const query = 'DHAKA';
const city = 'Dhaka';
console.log(city.toLowerCase() === query.toLowerCase());

console.log('I'.toLowerCase());
console.log('I'.toLocaleLowerCase('tr'));
```

Output

```
DHAKA
dhaka
ঢাকা
true
ı
ɪ
```

বড়-ছোট হাতের অক্ষর নির্বিশেষে দুটি লেখা মেলাতে **দুই পাশকেই** একই রূপে আনুন, তারপর তুলনা করুন। শুধু এক পাশ বদলালে কাজ হয় না।

বাংলা লিপিতে বড়-ছোট হাতের অক্ষরের ধারণা নেই, তাই `toUpperCase()` বাংলা লেখা অপরিবর্তিত রাখে। শেষ দুই লাইন দেখায় যে সব ভাষায় নিয়ম এক নয়: তুর্কি ভাষায় `I`-এর ছোট হাতের রূপ বিন্দুবিহীন `ɪ`। ব্যবহারকারীর ভাষা জানা থাকলে `toLocaleLowerCase()` ব্যবহার করুন।

ক্রম অনুযায়ী সাজানো

অধ্যায় ৩-এ দেখেছেন, `<` দিয়ে String-এর তুলনা হয় Unicode নম্বর ধরে, যা মানুষের অভিধানের ক্রম নয়। ভাষার নিয়ম মেনে সাজাতে `localeCompare()` ব্যবহার করুন।

locale-compare.js · JavaScript চলবে: Node.js

```
const words = ['cherry', 'Banana', 'apple'];

console.log([...words].sort());
console.log([...words].sort((a, b) => a.localeCompare(b)));
```

Output

```
[ 'Banana', 'apple', 'cherry' ]
[ 'apple', 'Banana', 'cherry' ]
```

সাধারণ `sort()` Unicode নম্বর ধরে সাজায়, আর সেখানে সব বড় হাতের অক্ষর ছোট হাতের অক্ষরের আগে; তাই `'Banana'` প্রথমে এসেছে। `localeCompare()` মানুষের অভিধানের মতো সাজিয়েছে। বাংলা নাম সাজাতে `a.localeCompare(b, 'bn')` লিখে ভাষাটিও বলে দিতে পারেন।

৮.৮ split এবং join: শব্দে ভাগ করা

`split()` একটি String-কে নির্দিষ্ট বিভাজক ধরে ভেঙে Array তৈরি করে; `join()` (অধ্যায় ৬) উল্টো কাজ করে।

split-join.js · JavaScript চলবে: Node.js

```
console.log('আম,কলা,লিচু'.split(','));
console.log('a,b,,c'.split(','));
console.log('Sara  Ahmed'.split(' '));
console.log('2026-10-15'.split('-').map(Number));
console.log(['JavaScript', 'শিখুন'].join(' '));
```

Output

```
[ 'আম', 'কলা', 'লিচু' ]
[ 'a', 'b', '', 'c' ]
[ 'Sara', '', 'Ahmed' ]
[ 2026, 10, 15 ]
JavaScript শিখুন
```

- দুটি বিভাজক পাশাপাশি থাকলে মাঝখানে একটি ফাঁকা String তৈরি হয় (দ্বিতীয় ও তৃতীয় লাইন)।
- `split('-').map(Number)` একটি তারিখ-String থেকে সংখ্যার Array তৈরি করেছে। `Number` এখানে একটি Function হিসেবে `map`-এ পাঠানো হয়েছে (অধ্যায় ৫)।

> **সতর্কতা:** `split(',')` একটি পূর্ণাঙ্গ CSV Reader নয়। একটি ঘরের মানের ভেতরেই কমা থাকলে (`"Dhaka, Bangladesh"`) ভুল ভাগ হবে। বাস্তব CSV File-এর জন্য উপযুক্ত Library ব্যবহার করুন।

একটি লেখাকে শব্দে ভাগ করাকে **Tokenization** বলে। শুধু ফাঁকা জায়গা ধরে ভাগ করলে বিরামচিহ্ন শব্দের সঙ্গে লেগে থাকে (`'করুন।'`); নির্ভরযোগ্য Tokenization-এর জন্য Regular Expression লাগে, যা পরের দুই অনুচ্ছেদে আসছে।

৮.৯ Unicode Normalization

একই রকম দেখতে লেখা Unicode-এ একাধিক উপায়ে লেখা যায়। `é` একটি একক অক্ষর হতে পারে, আবার `e` এবং তার ওপরের একটি আলাদা চিহ্নের সমন্বয়ও হতে পারে। বাংলায়ও এমন আছে: `য়` একটি একক অক্ষর হিসেবে লেখা যায়, আবার `য` এবং নিচের বিন্দু (নুক্তা) হিসেবেও।

normalize.js · JavaScript — চলবে: Browser Console ও Node.js

```
const composed = '\u00E9';
const decomposed = 'e\u0301';

console.log(composed.length, decomposed.length);
console.log(composed === decomposed);
console.log(composed.normalize('NFC') === decomposed.normalize('NFC'));

const ya1 = '\u09DF';
const ya2 = '\u09AF\u09BC';

console.log(ya1 === ya2, ya1.length, ya2.length);
console.log(ya1.normalize('NFC') === ya2.normalize('NFC'));
```

Output

```
1 2
false
true
false 1 2
true
```

পর্দায় দুটি একই দেখায় (`é`), কিন্তু প্রথমটি একটি Code Point এবং দ্বিতীয়টি দুটি; ভেতরের Code Point আলাদা, তাই `===` মিথ্যা। একজন ব্যবহারকারী একটি Keyboard দিয়ে নাম লিখে নিবন্ধন করলেন, পরে অন্য Keyboard দিয়ে সেই নাম লিখে খুঁজলেন—Program বলবে "পাওয়া যায়নি"। বাংলা লেখা নিয়ে কাজ করা Program-এ এটি বাস্তব সমস্যা।

Normalization লেখাকে একটি নির্দিষ্ট আদর্শ রূপে আনে। `normalize('NFC')` সবচেয়ে প্রচলিত রূপ। নিয়ম: **তুলনা, খোঁজা বা সংরক্ষণের আগে দুই পাশকেই একই রূপে Normalize করুন।**

রূপ	অর্থ
NFC	যেখানে সম্ভব, অক্ষর ও চিহ্ন জোড়া লাগানো; সাধারণ কাজে এটিই
NFD	অক্ষর ও চিহ্ন আলাদা করা
NFKC / NFKD	এর সঙ্গে কিছু প্রতীককে সাধারণ অক্ষরে রূপান্তর; অর্থ বদলে যেতে পারে, তাই সাবধানে

একটি মজার ব্যতিক্রম: `য়`-এর ক্ষেত্রে Unicode-এর নিয়মে NFC জোড়া লাগানো একক রূপটি নয়, বরং `য` ও নুক্তার দুই-অংশের রূপটিকেই আদর্শ ধরে। তাতে আপনার কিছু করার নেই: দুই পাশ একই Function দিয়ে Normalize করলে তুলনা ঠিকই মেলে। কোন রূপটি আদর্শ, তা মুখস্থ রাখার দরকার নেই।

৮.১০ Regular Expression-এর পরিচয়

Regular Expression (সংক্ষেপে Regex) হলো লেখার একটি Pattern বা ছাঁচ বর্ণনা করার ভাষা। "যেকোনো সংখ্যা", "এক বা একাধিক ফাঁকা জায়গা", "শুরুতে 01, তারপর ৯টি সংখ্যা"—এমন বর্ণনা Regex দিয়ে লেখা যায়। দুটি Slash-এর মাঝে Pattern লেখা হয়, শেষে ঐচ্ছিক **Flag**।

regex-intro.js · JavaScript — চলবে: Node.js

```
const text = 'Order 12 এবং Order 345';

console.log(/\d+/.test(text));
console.log(text.match(/\d+/g));
console.log(text.replace(/\d+/g, '#'));
console.log('Sara    Ahmed'.replace(/\s+/g, ' '));
console.log(/^order/i.test(text));
```

Output

```
true
[ '12', '345' ]
Order # এবং Order #
Sara Ahmed
true
```

অংশ	অর্থ
`\d`	একটি সংখ্যা (০–৯, ইংরেজি)
`\s`	একটি ফাঁকা জায়গা (Space, Tab, নতুন লাইন)
`+`	আগের জিনিসটি এক বা একাধিকবার
`{n}`	আগের জিনিসটি ঠিক `n` বার
`^` / `$`	লেখার শুরু / শেষ
`[abc]`	এদের যেকোনো একটি
Flag `g`	সব মিল খোঁজা (Global)
Flag `i`	বড়-ছোট হাতের অক্ষর নির্বিশেষে

অংশ	অর্থ
Flag `u`	Unicode-সচেতন Pattern

- **`regex.test(text)`** — মিল আছে কি না।
- **`text.match(regex)`** — `g` Flag-সহ সব মিলের Array; কিছু না মিললে `null`।
- **`text.replace(regex, replacement)`** — `g` থাকলে সব মিল বদলায়। অনুচ্ছেদ ৮.৬-এর মাঝের বাড়তি ফাঁকা জায়গা `/\s+/g` দিয়ে একটিতে আনা হয়েছে।

বাংলা শব্দ খোঁজা

`\w` শুধু ইংরেজি অক্ষর, সংখ্যা ও `_` চেনে; বাংলা চেনে না। Unicode-সচেতন Pattern-এ `\p{L}` (যেকোনো ভাষার অক্ষর), `\p{M}` (কার-চিহ্ন, হসন্তের মতো যুক্ত-চিহ্ন) এবং `\p{N}` (সংখ্যা) ব্যবহার করা যায়। এগুলোর জন্য `u` Flag বাধ্যতামূলক।

unicode-words.js · JavaScript — চলবে: Node.js

```
const sentence = 'JavaScript শিখুন, প্রতিদিন ১০ মিনিট!';

console.log(sentence.match(/\w+/g));
console.log(sentence.match(/[\p{L}\p{M}\p{N}]+/gu));
```

Output

```
[ 'JavaScript' ]
[ 'JavaScript', 'শিখুন', 'প্রতিদিন', '১০', 'মিনিট' ]
```

`\w` বাংলা শব্দগুলো পুরোপুরি বাদ দিয়েছে। Unicode Pattern সব শব্দ এবং বাংলা সংখ্যাও পেয়েছে, আর বিরামচিহ্ন বাদ দিয়েছে। `\p{M}` বাদ দিলে কার-চিহ্নে এসে শব্দ ভেঙে যেত (`'শ'`, `'খ'`...)।

Regular Expression একটি বিস্তৃত বিষয়; বিস্তারিত আলোচনা অধ্যায় ১৭-এ। আপাতত জেনে রাখুন: Regex শক্তিশালী, কিন্তু জটিল Pattern পড়া ও ঠিক রাখা কঠিন। সহজ String Method দিয়ে কাজ হলে সেটিই ব্যবহার করুন।

৮.১১ Text যাচাই

ব্যবহারকারীর লেখা যাচাইয়ের সাধারণ ধাপ: Normalize ও `trim` করা, তারপর দৈর্ঘ্য, তারপর গঠন।

validation.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isBdMobile = (text) => /^01[3-9]\d{8}$/.test(text.trim());
const looksLikeEmail = (text) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(text.trim());

console.log(isBdMobile('01712345678'));
console.log(isBdMobile('0171234567'));
console.log(isBdMobile('02712345678'));
console.log(looksLikeEmail('sara@example.com'));
console.log(looksLikeEmail('sara@example'));
console.log(looksLikeEmail('sara @example.com'));
```

Output

```
true
false
false
true
false
false
```

- **মোবাইল নম্বর:** শুরুতে `01`, তারপর ৩ থেকে ৯-এর একটি সংখ্যা, তারপর ঠিক ৮টি সংখ্যা; `^` ও `$` নিশ্চিত করে যে পুরো লেখাটিই এই গঠনের, আগে-পরে বাড়তি কিছু নেই।
- **Email:** এই Pattern শুধু মোটা দাগের গঠন দেখে: `@`-এর আগে ও পরে কিছু আছে, এবং পরে একটি ডট। Email-এর পূর্ণ নিয়ম অত্যন্ত জটিল; কোনো Regex দিয়েই প্রমাণ হয় না যে ঠিকানাটি আসলে আছে। সেটি জানার একমাত্র উপায় সেখানে একটি যাচাই-বার্তা পাঠানো।

> **নোট:** উদাহরণের মোবাইল নম্বরের নিয়ম শেখানোর জন্য সরলীকৃত। বাস্তব Program-এ দেশের Code (`+880`) এবং মাঝের হাইফেন বা ফাঁকা জায়গা আগে সরিয়ে নিয়ে তবেই যাচাই করুন।

৮.১২ নিরাপদ Rendering: XSS

অধ্যায় ১-এ বলা হয়েছিল, ব্যবহারকারীর লেখা Page-এ দেখাতে `textContent` ব্যবহার করুন, `innerHTML` নয়। এবার কারণটি নিজের চোখে দেখুন। উদাহরণটি Browser-এ চালান।

xss-demo.js · JavaScript চলবে: Browser

```
const userText = 'Hello <img src="" onerror="console.log(\'XSS: Code চলে গেছে!\')">';

const unsafe = document.createElement('p');
unsafe.innerHTML = userText;

const safe = document.createElement('p');
safe.textContent = userText;

console.log(safe.innerHTML);
console.log(safe.children.length, unsafe.children.length);
```

Output

```
Hello &lt;img src="" onerror="console.log('XSS: Code চলে গেছে!')"&gt;
0 1
XSS: Code চলে গেছে!
```

ব্যবহারকারী একটি "নাম" লিখেছেন, যার ভেতরে একটি `<img>` Tag লুকানো। `innerHTML` লেখাটিকে HTML হিসেবে পড়েছে: একটি সত্যিকারের ছবি-Element তৈরি হয়েছে, ছবিটি Load হতে ব্যর্থ হয়েছে, এবং `onerror`-এর Code চলে গেছে। এখানে Code শুধু একটি বার্তা দেখিয়েছে; আক্রমণকারী একই জায়গায় ব্যবহারকারীর Session চুরি বা তাঁর পক্ষে কাজ করার Code বসাতে পারেন। এই আক্রমণের নাম **Cross-Site Scripting (XSS)**।

`textContent` একই লেখাকে সাধারণ লেখা হিসেবে রেখেছে: Element-এর ভেতরে কোনো Tag তৈরি হয়নি (`0`), এবং `<` ও `>` নিরাপদ রূপ `<` ও `>`-এ পরিণত হয়েছে।

খোঁজার শব্দ চিহ্নিত করা

Search Result-এ খোঁজার শব্দটি গাঢ় করে দেখানো একটি সাধারণ কাজ, এবং XSS-এর একটি সাধারণ উৎস: লেখার মাঝে `<mark>` জুড়ে দিয়ে `innerHTML`-এ বসানো। নিরাপদ উপায় হলো লেখাকে টুকরো করে প্রতিটি টুকরো `textContent` দিয়ে বসানো, এবং শুধু মিলে যাওয়া টুকরোর জন্য `<mark>` Element তৈরি করা। Project-এর `highlight` Function লেখাকে এমন টুকরোয় ভাগ করার কাজটিই করে; Page-এ বসানোর Code অধ্যায় ৯-এ আসবে।

নিরাপত্তা প্রেক্ষাপট-নির্ভর: HTML-এর ভেতরে যা নিরাপদ, URL, CSS বা Database Query-র ভেতরে তা নিরাপদ নয়। প্রতিটি প্রেক্ষাপটের নিয়ম অধ্যায় ২০-এ আলোচনা করা হয়েছে।

৮.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Method-এর ফল কোথাও না রাখা	String বদলায় না	`text = text.trim()`

ভুল	কী দেখা যায়	সমাধান
`replace()` দিয়ে সব বদলানোর চেষ্টা	শুধু প্রথমটি বদলায়	`replaceAll()` বা Regex-এ `g`
শুধু এক পাশ ছোট হাতের অক্ষরে এনে তুলনা	মিল পাওয়া যায় না	দুই পাশই একই রূপে আনুন
Normalization ছাড়া বাংলা লেখা তুলনা	একই দেখতে শব্দ "আলাদা"	দুই পাশেই `normalize('NFC')`
`length` দিয়ে বাংলা অক্ষর গোনা	প্রত্যাশার চেয়ে বেশি সংখ্যা	`Intl.Segmenter`
`slice` দিয়ে বাংলা লেখা কাটা	অসম্পূর্ণ যুক্তাক্ষর	Grapheme ধরে কাটুন
`\w` দিয়ে বাংলা শব্দ খোঁজা	বাংলা বাদ পড়ে	`/[\p{L}\p{M}\p{N}]+/gu`
Regex-এ `\p{...}` কিন্তু `u` Flag নেই	`SyntaxError` বা ভুল মিল	`u` Flag দিন
`match()`-এর ফল যাচাই ছাড়া ব্যবহার	মিল না থাকলে `null`-এর ওপর Error	`?? []`
ব্যবহারকারীর লেখা `innerHTML`-এ	XSS	`textContent`
`split(',')`-কে CSV Reader ভাবা	উদ্ধৃত মানের ভেতরের কমায় ভুল ভাগ	উপযুক্ত CSV Library

৮.১৪ Debugging নির্দেশিকা

লেখা-সংক্রান্ত সবচেয়ে বিভ্রান্তিকর Bug হলো দুটি লেখা দেখতে একই, কিন্তু Program বলে আলাদা। সমাধান: অক্ষরগুলোর Unicode নম্বর দেখা।

debug-text.js · JavaScript — চলবে: Browser Console ও Node.js

```
const showCodes = (text) =>
  [...text].map((c) => c.codePointAt(0).toString(16).padStart(4, '0')).join(' ');

console.log(showCodes('\u09DF'));
console.log(showCodes('\u09AF\u09BC'));
const typed = 'Sara\u00A0Ahmed ';

console.log(JSON.stringify('Sara '));
console.log(typed.split(' ').length);
console.log(showCodes(typed.slice(4, 5)));
```

Output

```
09df
09af 09bc
"Sara "
2
00a0
```

`codePointAt(0)` একটি অক্ষরের নম্বর দেয়, আর `toString(16)` সেটি Unicode-এর প্রচলিত ষোলোভিত্তিক রূপে দেখায়। এখন স্পষ্ট যে দুটি "য়" আলাদা। তৃতীয় লাইনে `JSON.stringify` লেখাটিকে Quotation Mark-এ ঘিরে দেখায়, ফলে শেষের ফাঁকা জায়গাটি চোখে পড়ে। কিন্তু `typed`-এর মাঝের ফাঁকা জায়গাটি আসলে একটি Non-breaking Space (`\u00A0`), যা সাধারণ Space-এর মতো দেখায়, এমনকি `JSON.stringify`-তেও। চতুর্থ লাইনে তা ধরা পড়েছে: দুটি Space থাকলে `split(' ')` তিনটি অংশ দিত, দিয়েছে দুটি। শেষ লাইনে `showCodes` অক্ষরটির আসল নম্বর `00a0` দেখিয়েছে। লুকানো অক্ষর ধরার নিশ্চিত উপায় এটিই।

লেখা নিয়ে কাজ প্রত্যাশামতো না হলে:

১. লেখাটি `JSON.stringify()` দিয়ে দেখুন: আগে-পরের ফাঁকা জায়গা এবং নতুন লাইনের মতো লুকানো অক্ষর ধরা পড়ে।
২. দেখতে একই দুটি লেখা না মিললে দুটির Code Point নম্বর মিলিয়ে দেখুন।
৩. তুলনা বা খোঁজার আগে দুই পাশে একই ক্রমে `normalize('NFC')`, `trim()` এবং `toLowerCase()` হয়েছে কি না দেখুন।
৪. সংখ্যা ভুল হলে দেখুন কী গুনছেন: Code Unit, Code Point, না Grapheme।
৫. Regex না মিললে একটি সরল লেখা দিয়ে আলাদাভাবে পরীক্ষা করুন, এবং Flag (`g`, `i`, `u`) মিলিয়ে নিন।
৬. Page-এ লেখার জায়গায় অপ্রত্যাশিত Element তৈরি হলে খুঁজুন কোথায় `innerHTML` ব্যবহার হয়েছে।

৮.১৫ পেশাদার অভ্যাস

- **বাইরের লেখা প্রথমেই পরিষ্কার করুন:** `normalize('NFC')`, তারপর `trim()`। সংরক্ষণের আগে একবার, যাতে পরে প্রতিবার না করতে হয়।
- **তুলনার জন্য একটি Function রাখুন,** যা Normalize, `trim` ও ছোট হাতের অক্ষরে আনার কাজ একসঙ্গে করে, এবং সব জায়গায় সেটিই ব্যবহার করুন।
- **গণনার ধরন সচেতনভাবে বেছে নিন:** সংরক্ষণের সীমা Code Unit-এ, মানুষের জন্য সীমা Grapheme-এ।
- **বাংলা লেখার জন্য Unicode-সচেতন Regex** (`u` Flag এবং `\p{...}`)।
- **সহজ Method আগে, Regex পরে।** `startsWith('01')` পড়তে `/^01/`-এর চেয়ে সহজ।
- **ব্যবহারকারীর লেখা সব সময় `textContent` দিয়ে।**
- **Template Literal দিয়ে লেখা গঠন করুন;** `+` দিয়ে লম্বা জোড়া লাগানোর চেয়ে পড়া সহজ।
- **Regex-কে নাম দিন বা Comment দিন;** `/^01[3-9]\d{8}$/` এক মাস পরে নিজেও বুঝতে কষ্ট হবে।

৮.১৬ Project: Content Search Utility

একজন লেখক বাংলা ও ইংরেজি মেশানো একটি লেখা তৈরি করছেন। তাঁর জন্য একটি সহায়ক Program লিখবেন: শব্দ ও অক্ষর গণনা, Web Address-এর জন্য Slug, একটি ছোট সারাংশ, সবচেয়ে বেশি ব্যবহৃত শব্দ, এবং লেখার ভেতরে একটি শব্দ খুঁজে চিহ্নিত করা।

Project-এর চাহিদা

- বাংলা, ইংরেজি এবং মেশানো লেখা সঠিকভাবে সামলাতে হবে।
- সব তুলনার আগে লেখা Unicode NFC-তে আনা হবে, এবং খোঁজা হবে বড়-ছোট হাতের অক্ষর নির্বিশেষে।
- অক্ষর গণনায় Code Point এবং চোখে দেখা অক্ষর (Grapheme)—দুটোই দেখাতে হবে।
- Slug: ছোট হাতের অক্ষর; শব্দগুলো হাইফেন দিয়ে জোড়া; বিরামচিহ্ন বাদ; বাংলা অক্ষর থাকবে।
- সারাংশ: নির্দিষ্ট দৈর্ঘ্যের মধ্যে, কোনো শব্দ মাঝখানে না কেটে, শেষে `...`।
- সবচেয়ে বেশি ব্যবহৃত শব্দ: সংখ্যাসহ, সমান সংখ্যার ক্ষেত্রে বর্ণানুক্রমে; খুব ছোট শব্দ বাদ।
- খোঁজার শব্দের সব অবস্থান, এবং লেখায় সেগুলো `[ ]` দিয়ে চিহ্নিত করা। কোনো HTML তৈরি হবে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
content-search-utility/
├── text-tools.js
├── README.md
└── tests/
    ├── load-script.js
    └── text-tools.test.js
```

text-tools.js

File-টি তিন অংশে দেখানো হলো; তিনটি মিলে সম্পূর্ণ File। অংশগুলো পরপর একই File-এ লিখুন। `// ...` লাইনগুলো শুধু অংশের সংযোগ বোঝায়; সেগুলো লিখতে হবে না।

অংশ ১: Normalization, শব্দ ও অক্ষর গণনা

text-tools.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const WORD = /[\p{L}\p{M}\p{N}]+/gu;

// তুলনার জন্য একই রূপ: NFC, ছোট হাতের অক্ষর, একটিমাত্র ফাঁকা জায়গা।
function normalizeText(text) {
  return String(text ?? '')
    .normalize('NFC')
    .toLowerCase()
    .replace(/\s+/g, ' ')
    .trim();
}

// লেখার শব্দগুলোর Array (বাংলা ও ইংরেজি)।
const toWords = (text) => normalizeText(text).match(WORD) ?? [];
```

```
// অক্ষর গণনা: Code Point এবং চোখে দেখা অক্ষর।
function countCharacters(text) {
  const clean = String(text ?? '').normalize('NFC');
  const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });

  return {
    codePoints: [...clean].length,
    graphemes: [...segmenter.segment(clean)].length,
  };
}

// ...
```

`normalizeText` অনুচ্ছেদ ৮.১৫-এর "তুলনার জন্য একটি Function"। চারটি ধাপ একটি Chain-এ: প্রতিটি Method নতুন String ফেরত দেয়, তাই পরের Method সরাসরি তার ওপর Call করা যায় (অধ্যায় ৬-এর Method Chaining)। `String(text ?? '')` নিশ্চিত করে যে `null` বা সংখ্যা এলেও কাজ চলবে।

`toWords`-এ `match` মিল না পেলে `null` দেয়, তাই `?? []`। ফলে ফাঁকা লেখায়ও সব সময় একটি Array পাওয়া যায়।

অংশ ২: Slug, সারাংশ এবং শব্দের ঘনত্ব

text-tools.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// URL-এর উপযোগী Slug: শব্দগুলো হাইফেন দিয়ে জোড়া।
const makeSlug = (title) => toWords(title).join('-');

// শব্দ না কেটে নির্দিষ্ট দৈর্ঘ্যের সারাংশ।
function makeExcerpt(text, maxLength = 60) {
  const clean = String(text ?? '').normalize('NFC').replace(/\s+/g, ' ').trim();

  if (clean.length <= maxLength) {
    return clean;
  }

  const cut = clean.slice(0, maxLength);
  const lastSpace = cut.lastIndexOf(' ');
  const base = lastSpace > 0 ? cut.slice(0, lastSpace) : cut;

  return `${base}...`;
}

// সবচেয়ে বেশি ব্যবহৃত শব্দ, কমপক্ষে minLength অক্ষরের।
function topWords(text, limit = 5, minLength = 3) {
  const counts = {};

  for (const word of toWords(text)) {
    if ([...word].length >= minLength) {
```

```
      counts[word] = (counts[word] ?? 0) + 1;
    }
  }

  return Object.entries(counts)
    .sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]))
    .slice(0, limit)
    .map(([word, count]) => `${word} (${count})`);
}

// ...
```

`makeSlug` আলাদা কোনো নিয়ম লেখেনি: `toWords` ইতিমধ্যে ছোট হাতের অক্ষরে আনে এবং বিরামচিহ্ন বাদ দেয়, তাই শুধু হাইফেন দিয়ে জোড়া লাগানোই যথেষ্ট। পুনর্ব্যবহারের সুফল এটিই।

`makeExcerpt` প্রথমে নির্দিষ্ট দৈর্ঘ্যে কাটে, তারপর শেষ ফাঁকা জায়গা পর্যন্ত পিছিয়ে আসে, যাতে শেষ শব্দটি অসম্পূর্ণ না থাকে। শব্দের মাঝে কাটলে বাংলায় যুক্তাক্ষর ভেঙে যেতে পারত (অনুচ্ছেদ ৮.৪); শব্দের সীমায় কাটলে সেই ঝুঁকি থাকে না।

`topWords` অধ্যায় ৭-এর গণনা-Object ধরন ব্যবহার করে: শব্দটি Key, সংখ্যা Value। তারপর `Object.entries` থেকে Array, এবং দুই স্তরের সাজানো: `b[1] - a[1]` বেশি সংখ্যা আগে আনে; সংখ্যা সমান হলে বিয়োগফল `0`, যা Falsy, তাই `||`-এর ডান পাশের `localeCompare` বর্ণানুক্রমে সাজায় (অধ্যায় ৩-এর Short-Circuit)।

অংশ ৩: খোঁজা, চিহ্নিত করা এবং উদাহরণ

text-tools.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// খোঁজার শব্দের সব অবস্থান (NFC লেখায় Index), বড়-ছোট হাতের অক্ষর নির্বিশেষে।
function findAll(text, query) {
  const haystack = String(text).normalize('NFC').toLowerCase();
  const needle = normalizeText(query);
  const positions = [];

  if (needle === '') {
    return positions;
  }

  let index = haystack.indexOf(needle);

  while (index !== -1) {
    positions.push(index);
    index = haystack.indexOf(needle, index + needle.length);
  }

  return positions;
}
```

```
// প্রতিটি মিল [ ] দিয়ে চিহ্নিত করে; HTML তৈরি করে না।
function highlight(text, query) {
  const clean = String(text).normalize('NFC');
  const size = normalizeText(query).length;
  let result = '';
  let last = 0;

  for (const index of findAll(clean, query)) {
    result += `${clean.slice(last, index)}[${clean.slice(index, index + size)}]`;
    last = index + size;
  }

  return result + clean.slice(last);
}

const article = `JavaScript শেখা শুরু করুন। JavaScript দিয়ে Browser ও Server
দুই জায়গাতেই কাজ করা যায়। প্রতিদিন একটু করে JavaScript অনুশীলন করুন।`;

const { codePoints, graphemes } = countCharacters('ক্ষমা');

console.log(`শব্দ: ${toWords(article).length}`);
console.log(`ক্ষমা: ${codePoints} Code Point, ${graphemes} অক্ষর`);
console.log(makeSlug('JavaScript শেখার প্রথম ধাপ!'));
console.log(makeExcerpt(article, 40));
console.log(topWords(article, 3).join(', '));
console.log(findAll(article, 'JAVASCRIPT').join(', '));
console.log(highlight('প্রতিদিন অনুশীলন করুন, নিয়মিত অনুশীলন করুন।', 'অনুশীলন'));
```

Output

```
শব্দ: 20
ক্ষমা: 5 Code Point, 2 অক্ষর
javascript-শেখার-প্রথম-ধাপ
JavaScript শেখা শুরু করুন। JavaScript...
javascript (3), করুন (2), browser (1)
0, 27, 107
প্রতিদিন [অনুশীলন] করুন, নিয়মিত [অনুশীলন] করুন।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **`findAll`-এর Loop।** `indexOf`-এর দ্বিতীয় Argument দিয়ে প্রতিবার আগের মিলের পর থেকে খোঁজা শুরু হয়; কিছু না পেলে `-1`, এবং `while` থামে। এটি অধ্যায় ৪-এর সেই `while`, যার শেষ আগে থেকে জানা নেই। প্রতিটি ধাপে `index` সামনে এগোয়, তাই Loop অবশ্যই শেষ হয়।
- **দুই পাশ একই রূপে।** লেখাটিকে NFC ও ছোট হাতের অক্ষরে আনা হয়েছে, খোঁজার শব্দটিকেও; তাই `'JAVASCRIPT'` তিনটি মিলই পেয়েছে।

- **`highlight` লেখাকে টুকরো করে।** প্রতিটি মিলের আগের অংশ, তারপর `[মিল]`, শেষে বাকি অংশ। মূল লেখার অক্ষরগুলোই বসানো হয়েছে (খোঁজার শব্দ নয়), তাই বড়-ছোট হাতের অক্ষর লেখায় যেমন ছিল তেমনই থাকে। Page-এ একই টুকরোগুলো `textContent` এবং `<mark>` Element দিয়ে বসালে XSS-এর ঝুঁকি থাকে না।
- **গণনা।** ২০টি শব্দ, কারণ বিরামচিহ্ন (`।`) শব্দ হিসেবে গোনা হয়নি। `ক্ষমা`-য় ৫টি Code Point কিন্তু চোখে ২টি অক্ষর।
- **সারাংশ।** ৪০ অক্ষরের সীমায় শেষ শব্দটি না কেটে পিছিয়ে এসে `...` যোগ হয়েছে।
- **শব্দের ঘনত্ব।** `javascript` তিনবার, `করুন` দুবার; বাকি সব একবার, তাই বর্ণানুক্রমে প্রথমটি, `browser`, এসেছে। তিন অক্ষরের কম শব্দ (`ও`) বাদ পড়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node text-tools.js
```

Call	যা পাওয়ার কথা
`makeSlug('JavaScript শেখার প্রথম ধাপ!')`	`javascript-শেখার-প্রথম-ধাপ`
`makeSlug('  Hello,  World  ')`	`hello-world`
`makeExcerpt('ছোট লেখা', 60)`	পুরো লেখাটিই, `...` ছাড়া
`countCharacters('ক্ষমা')`	Code Point 5, অক্ষর 2
`findAll(text, '')`	ফাঁকা তালিকা; Loop চলে না
`findAll('\u09DF', '\u09AF\u09BC')`	`[0]`; দুই রূপের য় মেলে
`highlight('Hello <b>', 'b')`	`Hello <[b]>`; কোনো HTML Element তৈরি হয় না

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে, যার কয়েকটি বিশেষভাবে বাংলা লেখা ও Normalization পরীক্ষা করে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
বাংলা শব্দ গোনা হয় না	Regex-এ `\w`	`/[\p{L}\p{M}\p{N}]+/gu`
শব্দগুলো কার-চিহ্নে ভেঙে যায়	Pattern-এ `\p{M}` নেই	`\p{M}` যোগ করুন
`SyntaxError: Invalid regular expression`	`\p{...}` আছে কিন্তু `u` Flag নেই	`u` Flag
খোঁজার শব্দ লেখায় থাকা সত্ত্বেও পাওয়া যায় না	এক পাশ Normalize বা ছোট হাতের অক্ষরে আনা হয়নি	দুই পাশে একই Function

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Program আটকে যায়	`while`-এ `index` এগোচ্ছে না, বা ফাঁকা শব্দে খোঁজা	`index + needle.length`; ফাঁকা শব্দে আগেই `return`
`TypeError: ... reading 'length'`	`match` থেকে `null`	`?? []`
সমান সংখ্যার শব্দের ক্রম প্রতিবার আলাদা	দ্বিতীয় স্তরের সাজানো নেই	`|| a[0].localeCompare(b[0])`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। খোঁজা আক্ষরিক: বানানভেদ (যেমন "শেখা" ও "শিখা"), শব্দের রূপভেদ বা সমার্থক শব্দ ধরে না। কিছু ভাষায় ছোট হাতের অক্ষরে আনলে লেখার দৈর্ঘ্য বদলে যায়; তখন চিহ্নিত করার অবস্থান সামান্য সরে যেতে পারে (বাংলা ও ইংরেজিতে এই সমস্যা হয় না)। Slug-এ বাংলা অক্ষর থাকে; কিছু পুরোনো Web System শুধু ইংরেজি Slug গ্রহণ করে। লেখা Program-এর ভেতরে দেওয়া; File থেকে পড়া হয় না।

৮.১৭ অনুশীলনী

প্রাথমিক স্তর

১. `'  JavaScript Mastery  '` থেকে আগে-পরের ফাঁকা জায়গা বাদ দিয়ে বড় হাতের অক্ষরে দেখান। মূল String-টি যে অপরিবর্তিত, তাও দেখান।

২. `'2026-10-15'` থেকে `slice` দিয়ে বছর, মাস ও দিন আলাদা করে দেখান।

৩. `'01712345678'`-এর শেষ তিনটি সংখ্যা ছাড়া বাকিগুলো `*` দিয়ে ঢেকে দেখান (`********678`)। `slice`, `repeat` বা `padStart` ব্যবহার করুন।

৪. না চালিয়ে বলুন: `'a-b-c'.replace('-', '+')` এবং `'a-b-c'.replaceAll('-', '+')`-এর ফল কী।

৫. নিচের Code-এ নাম মেলার কথা, কিন্তু মিলছে না। ভুলটি ঠিক করুন।

compare-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const saved = 'Sara Ahmed';
const typed = '  sara ahmed ';

console.log(saved.toLowerCase() === typed);
```

মধ্যম স্তর

৬. `countVowelSigns` নামে একটি Function লিখুন, যা একটি বাংলা লেখায় কয়টি আ-কার (`া`) আছে তা গোনে।

৭. `initials` নামে একটি Function লিখুন, যা পুরো নাম থেকে প্রতিটি অংশের প্রথম অক্ষর বড় হাতের করে ফেরত দেয় (`'sara ahmed khan'` থেকে `'SAK'`)। নামের মাঝে একাধিক ফাঁকা জায়গা থাকলেও যেন কাজ করে।

৮. `sameText` নামে একটি Function লিখুন, যা দুটি লেখাকে NFC, `trim` ও ছোট হাতের অক্ষরে এনে তুলনা করে। `'\u09DF'` ও `'\u09AF\u09BC'` দিয়ে পরীক্ষা করুন।

৯. `isValidUsername` নামে একটি Function লিখুন: ৩ থেকে ১৫ অক্ষর, শুধু ইংরেজি ছোট হাতের অক্ষর, সংখ্যা বা `_`, এবং অক্ষর দিয়ে শুরু।

চ্যালেঞ্জ স্তর

১০. **অক্ষর ধরে সারাংশ:** `shortenByLetters` নামে একটি Function লিখুন, যা `Intl.Segmenter` দিয়ে লেখাকে নির্দিষ্ট সংখ্যক চোখে-দেখা অক্ষরে কাটে, যাতে কোনো যুক্তাক্ষর না ভাঙে।

১১. **বাক্য গণনা:** `countSentences` নামে একটি Function লিখুন, যা দাঁড়ি (`।`), প্রশ্নবোধক (`?`) ও বিস্ময়সূচক (`!`) চিহ্ন ধরে বাক্য গোনে; ফাঁকা অংশ বাদ দেবে।

১২. **খোঁজার সারসংক্ষেপ:** Project-এর `findAll` ব্যবহার করে একটি Function লিখুন, যা খোঁজার শব্দ কতবার পাওয়া গেছে এবং প্রথম মিলের আশেপাশের ১০টি করে অক্ষর দেখায়।

৮.১৮ অনুশীলনীর সমাধান

সমাধান ১

trim-upper.js · JavaScript — চলবে: Browser Console ও Node.js

```
const title = '  JavaScript Mastery  ';
const clean = title.trim().toUpperCase();

console.log(`[${clean}]`);
console.log(`[${title}]`);
```

Output

```
[JAVASCRIPT MASTERY]
[  JavaScript Mastery  ]
```

সমাধান ২

date-parts.js · JavaScript — চলবে: Browser Console ও Node.js

```
const date = '2026-10-15';

console.log(date.slice(0, 4));
console.log(date.slice(5, 7));
console.log(date.slice(8));
```

Output

```
2026
10
15
```

সমাধান ৩

mask.js · JavaScript — চলবে: Browser Console ও Node.js

```
const phone = '01712345678';
const visible = phone.slice(-3);

console.log('*'.repeat(phone.length - 3) + visible);
console.log(visible.padStart(phone.length, '*'));
```

Output

```
********678
********678
```

দুটি পদ্ধতিই একই ফল দেয়। দ্বিতীয়টি ছোট: শেষ তিনটি অক্ষরের বাঁ দিকে `*` বসিয়ে মূল দৈর্ঘ্যে আনা।

সমাধান ৪ — `'a+b-c'` এবং `'a+b+c'`। `replace` শুধু প্রথম মিল বদলায়; `replaceAll` সব।

সমাধান ৫ — শুধু এক পাশ ছোট হাতের অক্ষরে আনা হয়েছে, আর `typed`-এর আগে-পরে ফাঁকা জায়গা আছে। দুই পাশেই একই কাজ করতে হবে।

compare-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const saved = 'Sara Ahmed';
const typed = '  sara ahmed ';
const clean = (text) => text.trim().toLowerCase();

console.log(clean(saved) === clean(typed));
```

Output

```
true
```

সমাধান ৬

vowel-signs.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countVowelSigns(text) {
  return [...text.normalize('NFC')].filter((c) => c === 'া').length;
}

console.log(countVowelSigns('আমার নাম সারা'));
```

Output

```
4
```

`আ` (স্বরবর্ণ) এবং `া` (আ-কার) আলাদা Code Point; এখানে শুধু কার-চিহ্নগুলো গোনা হয়েছে: মা, না, সা, রা।

সমাধান ৭

initials.js · JavaScript — চলবে: Browser Console ও Node.js

```
function initials(fullName) {
  return fullName
    .trim()
    .split(/\s+/)
    .map((part) => part[0].toUpperCase())
    .join('');
}

console.log(initials('sara ahmed khan'));
console.log(initials('  mina    das '));
```

Output

```
SAK
MD
```

`split(' ')`-এর বদলে `split(/\s+/)` ব্যবহার করায় একাধিক ফাঁকা জায়গায় ফাঁকা অংশ তৈরি হয়নি।

সমাধান ৮

same-text.js · JavaScript চলবে: Browser Console ও Node.js

```
function sameText(a, b) {
  const clean = (text) => text.normalize('NFC').trim().toLowerCase();
  return clean(a) === clean(b);
}

console.log('\u09DF' === '\u09AF\u09BC');
console.log(sameText('\u09DF', '\u09AF\u09BC'));
console.log(sameText(' Dhaka ', 'DHAKA'));
```

Output

```
false
true
true
```

সমাধান ৯

username.js · JavaScript চলবে: Browser Console ও Node.js

```
const isValidUsername = (name) => /^[a-z][a-z0-9_]{2,14}$/.test(name);

console.log(isValidUsername('sara_88'));
console.log(isValidUsername('8sara'));
console.log(isValidUsername('sa'));
console.log(isValidUsername('Sara'));
```

Output

```
true
false
false
false
```

প্রথম অক্ষর `[a-z]`, তারপর আরও ২ থেকে ১৪টি অনুমোদিত অক্ষর; মোট ৩ থেকে ১৫। `^` ও `$` পুরো নামটিকে নিয়মের মধ্যে রাখে।

সমাধান ১০

shorten-letters.js · JavaScript — চলবে: Browser Console ও Node.js

```
function shortenByLetters(text, count) {
  const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
  const letters = [...segmenter.segment(text)].map((part) => part.segment);

  if (letters.length <= count) {
    return text;
  }

  return `${letters.slice(0, count).join('')}...`;
}

console.log(shortenByLetters('ক্ষমা চাই', 2));
console.log('ক্ষমা চাই'.slice(0, 2));
```

Output

```
ক্ষমা...
ক্
```

দ্বিতীয় লাইন দেখায় সাধারণ `slice` কী করত: যুক্তাক্ষরের অর্ধেক।

সমাধান ১১

sentences.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countSentences(text) {
  return text
    .split(/[।?!]/)
    .filter((part) => part.trim() !== '').length;
}

console.log(countSentences('আমি শিখছি। আপনি? খুব ভালো!'));
console.log(countSentences(''));
```

Output

```
3
0
```

সমাধান ১২

search-summary.js · JavaScript — চলবে: Browser Console ও Node.js

```
function findAll(text, query) {
  const haystack = text.normalize('NFC').toLowerCase();
  const needle = query.normalize('NFC').toLowerCase().trim();
  const positions = [];
  let index = needle === '' ? -1 : haystack.indexOf(needle);

  while (index !== -1) {
    positions.push(index);
    index = haystack.indexOf(needle, index + needle.length);
  }

  return positions;
}

function searchSummary(text, query) {
  const positions = findAll(text, query);

  if (positions.length === 0) {
    return 'পাওয়া যায়নি';
  }

  const start = Math.max(0, positions[0] - 10);
  const around = text.slice(start, positions[0] + query.length + 10);

  return `${positions.length} বার পাওয়া গেছে: "...${around}..."`;
}

console.log(searchSummary('Node.js দিয়ে Server চালান, Node.js শিখুন', 'node'));
console.log(searchSummary('কিছু লেখা', 'JavaScript'));
```

Output

```
2 বার পাওয়া গেছে: "...Node.js দিয়ে ..."
পাওয়া যায়নি
```

`Math.max(0, ...)` নিশ্চিত করে যে মিলটি লেখার শুরুতে থাকলে Index ঋণাত্মক হয়ে শেষ থেকে গোনা শুরু না হয়।

৮.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `let s = 'abc'; s[0] = 'X';`-এর পর `s` কী?

ক) `'Xbc'`

খ) একটি Error

গ) `'abc'`

ঘ) `'X'`

প্রশ্ন ২. `'\u{1F600}'.length`-এর মান কত? (এটি একটি হাসিমুখ Emoji!)

ক) `2`

খ) `1`

গ) `4`

ঘ) `0`

প্রশ্ন ৩. একটি বাংলা শব্দে চোখে দেখা অক্ষর গুনতে কোনটি উপযুক্ত?

ক) `text.length`

খ) `[...text].length`

গ) `text.split('').length`

ঘ) `Intl.Segmenter`

প্রশ্ন ৪. `'a-b-c'.replace('-', '+')`-এর ফল কী?

ক) `'a+b+c'`

খ) `'a+b-c'`

গ) `'a-b+c'`

ঘ) `'abc'`

প্রশ্ন ৫. `'JavaScript'.includes('script')`-এর ফল কী?

ক) `false`, কারণ খোঁজা Case-sensitive

খ) `true`

গ) `4`

ঘ) `-1`

প্রশ্ন ৬. দেখতে একই দুটি বাংলা শব্দ `===` দিয়ে মিলছে না। প্রথমে কী করা উচিত?

ক) দুই পাশেই `normalize('NFC')` করা

খ) দুই পাশেই `toUpperCase()` করা

গ) `==` ব্যবহার করা

ঘ) `length` তুলনা করা

প্রশ্ন ৭. `'a,b,,c'.split(',')` কয়টি Element-এর Array দেয়?

ক) ৩

খ) ২

গ) ৪

ঘ) ৫

প্রশ্ন ৮. বাংলা শব্দ খুঁজতে কোন Pattern ঠিক?

ক) `/\w+/g`

খ) `/[a-z]+/gi`

গ) `/\d+/g`

ঘ) `/[\p{L}\p{M}\p{N}]+/gu`

প্রশ্ন ৯. `/^01[3-9]\d{8}$/` কোনটিকে গ্রহণ করে?

ক) `'01712345678'`

খ) `'0171234567'`

গ) `'+8801712345678'`

ঘ) `'01212345678'`

প্রশ্ন ১০. ব্যবহারকারীর লেখা Page-এ নিরাপদে দেখানোর উপায় কোনটি?

ক) `element.innerHTML = text`

খ) `element.textContent = text`

গ) `document.write(text)`

ঘ) `element.outerHTML = text`

৮.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–ক · ৬–ক · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	String Immutable; Index-এ Assignment নীরবে উপেক্ষিত হয়।
২	ক	`length` UTF-16 Code Unit গোনে; এই Emoji দুটি Code Unit।
৩	ঘ	বাংলায় একটি চোখে-দেখা অক্ষর একাধিক Code Point হতে পারে; Grapheme গোনে শুধু `Intl.Segmenter`।
৪	খ	`replace` শুধু প্রথম মিলটি বদলায়।
৫	ক	`'Script'` আর `'script'` আলাদা; দুই পাশ ছোট হাতের অক্ষরে এনে খুঁজতে হয়।
৬	ক	একই অক্ষর ভিন্ন Code Point-এ লেখা থাকতে পারে; Normalization দুই পাশকে একই রূপে আনে।
৭	গ	দুটি পাশাপাশি কমার মাঝে একটি ফাঁকা String: `'a'`, `'b'`, `''`, `'c'`।
৮	ঘ	`\w` বাংলা চেনে না; `\p{M}` কার-চিহ্ন রাখে; `u` Flag বাধ্যতামূলক।
৯	ক	শুরুতে `01`, তারপর ৩–৯-এর একটি সংখ্যা এবং ঠিক ৮টি সংখ্যা; মোট ১১টি।
১০	খ	`textContent` লেখাকে HTML হিসেবে পড়ে না, তাই ভেতরের Tag বা Code চলে না।

৮.২১ অধ্যায়ের সারসংক্ষেপ

- String Immutable; প্রতিটি Method নতুন String ফেরত দেয়।
- Template Literal মান বসাতে ও একাধিক লাইনে লিখতে; Escape Sequence বিশেষ অক্ষরের জন্য।
- `length` Code Unit গোনে; `[...text]` Code Point; চোখে দেখা অক্ষর `Intl.Segmenter`। বাংলায় তিনটি সংখ্যা আলাদা হতে পারে।
- পড়া: `at`, `slice`। খোঁজা: `includes`, `startsWith`, `endsWith`, `indexOf`; সবই Case-sensitive।
- পরিষ্কার: `trim`; বদলানো: `replace` (প্রথমটি), `replaceAll` (সব); সাজানো: `padStart`, `repeat`।
- তুলনার আগে দুই পাশ একই রূপে: `normalize('NFC')`, `trim()`, `toLowerCase()`।
- `split` String থেকে Array, `join` Array থেকে String।
- Regular Expression Pattern খোঁজে, বদলায় ও যাচাই করে; বাংলার জন্য `\p{L}\p{M}\p{N}` এবং `u` Flag।
- Regex দিয়ে গঠন যাচাই করা যায়, কিন্তু Email-এর মতো তথ্যের সত্যতা নয়।
- ব্যবহারকারীর লেখা `innerHTML`-এ বসালে XSS; সব সময় `textContent`।

পরের অধ্যায়ে DOM এবং Event: Page-এর Element তৈরি, বদলানো ও সরানো, ব্যবহারকারীর কাজে সাড়া দেওয়া, এবং এই অধ্যায়ের নিরাপদ Rendering-এর নিয়ম বাস্তবে প্রয়োগ।

৮.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Immutable	তৈরির পর বদলানো যায় না।
Template Literal	Backtick-এর String, যাতে `${ }` দিয়ে মান বসানো যায়।
Escape Sequence	`\` দিয়ে শুরু হওয়া বিশেষ অক্ষরের লিখিত রূপ।
Code Unit	JavaScript String-এর সংরক্ষণের একক (UTF-16)।
Code Point	Unicode-এর একটি অক্ষর-নম্বর।
Grapheme	পাঠকের চোখে একটি অক্ষর।
Unicode Normalization	একই লেখার ভিন্ন ভিন্ন Code Point-রূপকে একটি আদর্শ রূপে আনা।
Tokenization	লেখাকে শব্দে ভাগ করা।
Slug	Web Address-এর উপযোগী, হাইফেন দিয়ে জোড়া শব্দ।
Regular Expression	লেখার Pattern বর্ণনার ভাষা।
Flag	Regex-এর আচরণ বদলানোর চিহ্ন: `g`, `i`, `u`।
XSS	ব্যবহারকারীর লেখার মাধ্যমে Page-এ ক্ষতিকর Code চালানোর আক্রমণ।

৮.২৩ আরও পড়ুন

- **MDN — String:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String` — সব String Method-এর তালিকা।
- **MDN — String.prototype.normalize():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/normalize` — Unicode Normalization।
- **MDN — Intl.Segmenter:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter` — চোখে দেখা অক্ষর ও শব্দে ভাগ করা।
- **MDN — Regular expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions` — Regex-এর নির্দেশিকা।
- **OWASP — Cross Site Scripting (XSS):** `https://owasp.org/www-community/attacks/xss/` — XSS আক্রমণের ব্যাখ্যা।